



3

ROS2 Packages

18

Servo Joints

10ms

Control Period

1

ESP32 MCU

18-DOF HUMANOID ROBOT

ARCHITECTURE & PACKAGE DOCUMENTATION

Package Roles · File Responsibilities · System Topology

System Overview

This document provides a comprehensive reference for the **18-DOF Humanoid Robot** software stack, developed at INSAT as part of the Génie Logiciel curriculum (2024–2025). The architecture is organized into four distinct layers, each encapsulated in its own package or firmware module.

Layer	Package / Module	Language	Role
Firmware	Firmware (ESP32)	C++ / Arduino	Servo PWM · IMU bridge · micro-ROS
Model	my_robot_description	URDF / Xacro	Kinematic geometry · Visuals · Joint limits
Launch	my_robot_bringup	Python (launch)	Simulation bringup · Controller init
Algorithm	humanoid_robot	C++ / ROS2	IK · Balance · Gait · IMU filter

Firmware — ESP32

Hardware layer · Servo PWM · IMU bridge · micro-ROS transport

HARDWARE LAYER

The firmware runs exclusively on the **ESP32 microcontroller** aboard the physical robot. It bridges the ROS2 host (laptop) with real hardware: it receives joint angle arrays over WiFi UDP, converts them into PCA9685 PWM pulses to drive 18 MG996R servos, and streams raw IMU data back via the micro-ROS DDS transport. This layer *never* executes on the simulation host.

■ include/ — Header Files

`joint_definition.h`

Maps every joint name (`r_hip_roll`, `l_knee_pitch` ...) to its PCA9685 channel index 0–17. Single source of truth — must match URDF and controllers.yaml exactly to avoid silent index mismatches.

`microros_transport.h`

Declares the WiFi UDP transport layer used by micro-ROS to speak DDS with the ROS2 host. Configures remote IP address, port number, and agent reconnect policy.

`mpu6050_raw.h`

I²C driver interface for the MPU6050 IMU. Exposes raw read functions returning accelerometer and gyroscope int16 values before any software filtering is applied.

`robot_config.h`

Compile-time hardware constants: PCA9685 I²C address, PWM frequency (50 Hz), per-servo min/max pulse widths, WiFi SSID/password. All hardware-specific magic numbers live here.

`servo_controller.h`

Declares the angle→PWM conversion API. Converts float degrees to PCA9685 tick counts and clamps outputs to per-joint limits defined in `robot_config.h`.

■ src/ — Implementation Files

`main.cpp`

Arduino-style `setup()` + `loop()`. Initialises the micro-ROS node, subscribes to `/joint_commands` (Float32[18]), publishes `/imu/raw`, and drives the 10 ms control tick.

`micro_ros_transport.cpp`

Implements UDP socket open/close/read/write callbacks invoked internally by micro-ROS. Handles agent timeout detection and automatic reconnection logic.

`mpu6050_raw.cpp`

I²C transaction implementation: wakes MPU6050, sets sample rate to 100 Hz, reads 14 bytes of accelerometer + gyroscope data per tick, and returns raw int16 values.

`servo_controller.cpp`

Implements the angle→PWM mapping. Writes all 18 PCA9685 channels in a single I²C burst per control tick to minimize per-servo latency jitter.

Topics

■ PUBLISHED `/imu` → `sensor_msgs/Imu` (raw MPU6050 at 100 Hz)

■ SUBSCRIBED /joint_commands → Float32[18] (one angle per servo)

my_robot_description

MODEL LAYER

Model layer · URDF geometry · Joint limits · RViz config · No runtime nodes

my_robot_description is a purely static package — it contains no executable nodes. Its sole purpose is to describe the robot's physical reality: kinematic chain, joint axes and limits, link masses and inertia tensors, collision geometry, and visualization colours. Every other package in the stack depends on this one; a joint name change here must propagate everywhere.

■ urdf/ — Robot Model

`humanoid.urdf.xacro`

Main robot definition. Declares all 18 joints with axis vectors, degree limits (min/max), parent/child link hierarchy, collision meshes, and inertia tensors. This is the single source of truth for the kinematic chain — Pinocchio loads this file to compute Jacobians.

`materials.xacro`

RGBA colour definitions for RViz visualization (torso blue, legs grey, feet dark ...). Keeps visual styling decoupled from kinematics. Has no effect in Gazebo, which uses its own material plugin.

■ rviz/ — Visualization Config

`robot.rviz`

Saved RViz session state: camera angle, active displays (RobotModel, TF, IMU, ZMP marker ...), fixed frame set to base_link. Loading this file opens RViz in a consistent debugging configuration.

■ launch/ — Standalone Visualization

`rviz.launch.py`

Launches robot_state_publisher (reads URDF, publishes /tf), joint_state_publisher_gui (interactive slider panel for manual joint testing), and RViz with robot.rviz pre-loaded. Use for URDF verification without any controllers running.

Consumed by: my_robot_bringup · humanoid_robot (IK joint names) · Pinocchio (Jacobian) · controller_manager

my_robot_bringup

Launch layer · Gazebo Harmonic · controller_manager · ros2_control bridge

LAUNCH LAYER

my_robot_bringup is the simulation entry point. It contains no control algorithms — its entire role is orchestration: load the URDF, spawn the robot in Gazebo Harmonic, start the *controller_manager* node that hosts all joint controllers, and bridge Gazebo topics to ROS2 topics. The GazeboSimSystem plugin connects the physics engine to ros2_control.

■ config/ — Controller Configuration

controllers.yaml

Configures all four ros2_control controllers: • leg_controller — claims joints 9–18 (10 joints, both legs) • arm_controller — claims joints 1–6 (both arms) • head_controller — claims joints 7–8 (pan + tilt) • joint_state_broadcaster — reads all 18 joints, publishes /joint_states at 50 Hz. Joint names here must exactly match humanoid.urdf.xacro.

■ launch/ — Simulation Bringup

my_robot_gazebo.launch

Full simulation bringup sequence in order: ① Load URDF → robot_state_publisher ② Launch Gazebo Harmonic with DART physics engine ③ Spawn robot entity into Gazebo world ④ Start controller_manager with GazeboSimSystem plugin ⑤ Activate all four controllers ⑥ Launch ros_gz_bridge for /imu and /joint_states topics

Topics

■ PUBLISHED	/joint_states → sensor_msgs/JointState @ 50 Hz
■ PUBLISHED	/imu → sensor_msgs/Imu @ 50 Hz
■ PUBLISHED	/tf · /tf_static
■ SUBSCRIBED	/leg_controller/commands → Float64[10]
■ SUBSCRIBED	/arm_controller/commands → Float64[6]
■ SUBSCRIBED	/head_controller/commands → Float64[2]

humanoid_robot

Algorithm layer · IK · IMU filter · Balance PID · Gait planning · Research code

ALGORITHM LAYER

humanoid_robot is the core research package — every node that *computes* something lives here. It implements IMU complementary filtering, ankle balance correction via PID, inverse kinematics (analytical geometric and planned Jacobian/DLS via Pinocchio), and the forthcoming ZMP preview controller and gait generator. This is where the IK solver comparison will be benchmarked for the thesis.

■ config/ — Tunable Parameters

balance_params.yaml	PID gains for the ankle balance controller: Kp, Ki, Kd for pitch and roll axes independently. Also contains deadband threshold (°) and maximum correction angle. Tune these first on the real robot.
gait_params.yaml	Walking pattern parameters: step_length (m), step_width (m), step_height (m), step_duration (s), double_support_ratio, number_of_steps. Fed into the ZMP preview controller and foot trajectory generator.
imu_params.yaml	Complementary filter coefficient alpha (typically 0.98), IMU frame orientation relative to torso_link, and gyroscope bias correction values calibrated at startup.
ik_params.yaml	Geometric constants: L1=0.1225 m (thigh), L2=0.1230 m (shank), hip_width=0.08 m, com_height=0.19 m, knee_min_deg=5.0. Controls which IK solver is active: analytical jacobian hybrid.
robot_params.yaml	Global constants shared across nodes: total mass, CoM height for LIPM model, control frequency (100 Hz), joint limit table mirroring Table 3.1 from the PFA.

■ src/ — Existing Nodes (Active)

imu_filter.cpp	Subscribes to /imu (raw MPU6050 at 50 Hz). Applies a complementary filter fusing accelerometer and gyroscope data to compute pitch and roll angles. Publishes /tilt (geometry_msgs/Vector3) consumed by balance_controller.
balance_controller.cpp	Subscribes to /tilt. Runs independent PID loops on pitch_error and roll_error (desired = 0°). Computes ankle pitch and ankle roll correction angles ($\delta\theta$) and adds them to the IK joint angles before publishing to /leg_controller/commands.
inverse_kinematics.cpp	Current: monolithic analytical geometric IK. Planned refactor: split into ik_node.cpp (ROS2 pub/sub) + analytical_ik.hpp (closed-form math) + jacobian_ik.hpp (DLS via Pinocchio). Subscribes to /com_trajectory and /foot_trajectory, publishes /joint_angles_ik at 100 Hz.

■ src/ — Planned Nodes (Phase 5+)

`zmp_preview_controller.cpp`

Implements Kajita (2003) preview control. Reads `/footstep_plan`, solves the LQI Riccati equation offline, then runs the preview loop online at 100 Hz. Publishes `/com_trajectory`.

`gait_generator.cpp`

Generates a footstep sequence from a high-level command (walk forward N steps). Outputs `/footstep_plan` as a PoseArray of ordered foot placements with timing metadata.

`foot_trajectory.cpp`

Generates smooth swing-phase foot paths between footsteps using 3-point interpolation (lift-off → apex → land). Publishes `/foot_trajectory` (PoseArray[2]) at 100 Hz.

`trajectory_smoother.cpp`

Rate-limits joint angle changes to $MAX_DELTA = 2.5^\circ/tick$ to protect MG996R gears. Sits between IK output and `/leg_controller/commands`. Prevents violent servo slams.

`ik_benchmark.cpp`

Research node. Runs both analytical and DLS Jacobian solvers on identical inputs. Measures `solve_time_us`, `position_error_mm`, manipulability index, near-singularity events. Logs results to CSV for thesis analysis and comparison figures.

■ **launch/ — Execution Profiles**

`balance.launch.py`

Minimal launch: `imu_filter` + `balance_controller` only. Use for standing balance tests without any walking. Does not start IK or gait nodes.

`walking.launch.py`

Full walking stack: `imu_filter` + `balance_controller` + `inverse_kinematics`. Includes `my_robot_bringup` for Gazebo simulation. Main launch file for walking experiments.

End-to-End Data Flow

The diagram below shows how data propagates from the high-level gait planner down to the physical actuators (or Gazebo simulation). Each arrow represents a ROS2 topic publish/subscribe edge.

```
gait_generator → /footstep_plan → zmp_preview_controller
zmp_preview_controller → /com_trajectory → inverse_kinematics
foot_trajectory → /foot_trajectory → inverse_kinematics
inverse_kinematics → /joint_angles_ik → balance_controller
imu_filter → /tilt → balance_controller
balance_controller → /joint_angles_balanced → trajectory_smoother
trajectory_smoother → /leg_controller/commands → controller_manager
controller_manager → Gazebo sim OR ESP32 → PCA9685 → MG996R servos
```

■ Critical Consistency Rule

The joint name `r_knee_pitch` (and every other joint name) must be **identical** in all four locations listed below. Any single mismatch causes **silent failures**: the controller claims no joint, IK sends angles to the wrong servo index, or Pinocchio builds a broken kinematic model.

Location	Entry	Consequence if wrong
humanoid.urdf.xacro	<joint name="r_knee_pitch">	Pinocchio builds wrong kinematic tree
controllers.yaml	joints: [... r_knee_pitch ...]	Controller claims no joint → silent drop
joint_definition.h	JOINT_R_KNEE_PITCH = 10	Wrong PCA9685 channel → wrong servo moves
analytical_ik.cpp	servo index mapping	IK sends angle to wrong joint index

This document was generated from the official architecture specification. For the latest version of each file, refer to the Git repository. All joint indices are 0-based and map directly to PCA9685 channel numbers.